

TRAVAUX PRATIQUES

StreamFlix

Concevoir et développer un catalogue de séries alimenté par une API REST

Module	Conception et développement d'interfaces web
Objectif	Concevoir et développer un site web complet en HTML, CSS et JavaScript natifs, dont tout le contenu provient d'une API REST publique
Durée indicative	3 h 30, en autonomie
Compétences travaillées	Réaliser une interface web statique. Développer une interface web dynamique. Intégrer des données issues d'une API
Ressources autorisées	Tous documents, accès internet, documentation technique
Matériel	Ordinateur avec accès internet, éditeur de code, navigateur
Technologies imposées	HTML5, CSS3, JavaScript ES6+ natifs. Aucun framework, aucune bibliothèque externe
Organisation	Travail individuel

1. Objectif du TP

À la fin de ce TP, vous avez livré un site de trois pages qui fonctionne sans installation, s'adapte du téléphone au grand écran, et dont aucune donnée n'est écrite en dur dans le HTML. Le catalogue, les fiches détaillées et les genres sont récupérés auprès d'une API publique au chargement des pages.

Le sujet reprend, dans le même ordre, les quatre parties du cours : structure, mise en forme, dynamisation, qualité. Chaque partie est autonome : si vous bloquez sur un point, passez au suivant et revenez ensuite.

Répartition indicative du temps

Partie	Durée indicative
Partie 1 : Structure et contenu du site (HTML5)	30 min
Partie 2 : Mise en forme, responsive et animations (CSS3)	50 min
Partie 3 : Dynamisation et consommation de l'API (JavaScript)	90 min
Partie 4 : Accessibilité, qualité et livrables	40 min
TOTAL	3 h 30

Conseil de méthode

Faites d'abord exister le site : trois pages en HTML, puis un premier appel réseau qui affiche une liste de titres bruts. Vous ne passez à la mise en forme qu'une fois que les données arrivent. Un site sobre qui fonctionne vaut toujours mieux qu'un site soigné qui n'affiche rien.

2. Contexte

StreamFlix est une jeune plateforme de recommandation de séries. Elle ne diffuse pas de vidéo : elle aide ses visiteurs à trouver quoi regarder, à consulter la fiche d'une série et à constituer une liste personnelle de programmes à voir.

L'équipe technique se compose aujourd'hui de deux développeurs back-end. L'API de production, qui gérera le catalogue interne et les comptes utilisateurs, est en cours de développement et ne sera pas livrée avant plusieurs semaines. Pour ne pas bloquer le chantier front-end, le lead developer met à votre disposition une API publique de recette, TVMaze, dont le contrat d'interface est proche de celui de la future API de production. Vous développez donc en vous appuyant sur cette API ; la bascule vers l'API réelle devra se faire par le simple changement de l'URL de base.

Vous intervenez en tant que développeur front-end et êtes chargé de livrer la première version publique du site.

Contraintes communiquées par le lead developer

- **Aucun framework ni bibliothèque externe**
- **Aucune étape de build** : le site doit fonctionner tel quel, servi par un simple serveur de fichiers statiques.
- **Le mobile est prioritaire** : la majorité du trafic attendu provient d'un smartphone.

- **Aucune page blanche** : si l'API est lente ou indisponible, l'utilisateur doit voir un état de chargement puis un message d'erreur explicite.
- **Accessibilité** : contrastes et navigation adaptés à tous types d'utilisateur.
- **Un seul point de configuration** : l'URL de base de l'API ne doit apparaître qu'à un seul endroit du code.

3. L'API de recette

URL de base : `https://api.tvmaze.com` (documentation : `https://www.tvmaze.com/api`)

Points d'entrée utiles

Méthode	Endpoint	Description
GET	<code>/shows?page=0</code>	Catalogue : 250 séries par page
GET	<code>/shows/{id}</code>	Fiche détaillée d'une série
GET	<code>/search/shows?q={texte}</code>	Recherche par titre, renvoie un tableau d'objets { score, show }
GET	<code>/shows/{id}/episodes</code>	Liste des épisodes d'une série
GET	<code>/shows/{id}/cast</code>	Distribution d'une série

Réponse de GET /shows/1

```
{
  "id": 1,
  "name": "Under the Dome",
  "type": "Scripted",
  "language": "English",
  "genres": ["Drama", "Science-Fiction", "Thriller"],
  "status": "Ended",
  "runtime": 60,
  "premiered": "2013-06-24",
  "rating": { "average": 6.6 },
  "network": { "name": "CBS", "country": { "name": "United States" } },
  "image": { "medium": "https://static.tvmaze.com/...", "original": "..." },
  "summary": "<p><b>Under the Dome</b> is the story of a small town...</p>"
}
```

Trois pièges dans ces données

image peut valoir null : prévoyez une affiche de remplacement. rating.average peut valoir null : affichez une mention explicite plutôt que la valeur brute. summary contient du HTML : traitez-le en connaissance de cause.

Envoi de la sélection

La future API de production exposera une route de création de sélection. En attendant, utilisez le service de recette suivant, qui accepte une requête POST et répond comme le fera l'API réelle.

URL : `https://jsonplaceholder.typicode.com/posts`

Charge utile attendue :

```
{
  "userId": 1,
  "date": "2026-09-21",
  "shows": [
    { "showId": 5, "priority": 1 },
    { "showId": 12, "priority": 2 }
  ]
}
```

À savoir

Ce service simule la création de la sélection. Il répond avec un identifiant mais n'enregistre rien : un appel ultérieur ne retrouvera pas votre envoi. Traitez la réponse comme un succès sans chercher à la relire.

Partie 1 : Structure et contenu du site (HTML5)

Le site comporte trois pages. Aucun contenu de série n'y est écrit en dur : le HTML ne fournit que la structure d'accueil des données.

Page	Rôle
<code>/index.html</code>	Catalogue : liste des séries, recherche, filtre par genre, tri
<code>/serie.html?id={id}</code>	Fiche détaillée : affiche, résumé, genres, note, ajout à la liste
<code>/liste.html</code>	Ma liste : récapitulatif, ordre de priorité, envoi de la sélection

Structure sémantique

- Chaque page s'appuie sur les régions `<header>`, `<nav>`, `<main>`, `<section>` et `<footer>`.

- Une carte série du catalogue est portée par un élément `<article>` et affiche l'affiche, le titre, le ou les genres, l'année et la note.
- Les attributs ARIA nécessaires sont posés dès le balisage : `aria-current` sur le lien de la page active, `aria-live` sur la zone qui annonce le nombre de résultats, `aria-expanded` sur le bouton du menu mobile.
- La hiérarchie des titres est cohérente : un seul `h1` par page, aucun niveau sauté.

Menu de navigation

- Le menu donne accès au Catalogue et à Ma liste, cette dernière portant un compteur du nombre de séries retenues.
- La page courante est signalée visuellement et pour les technologies d'assistance.
- Sous 768 px, le menu se replie derrière un bouton hamburger.

Formulaires

- Le champ de recherche du catalogue s'appuie sur les attributs de validation HTML5 natifs (`type`, `minlength`, `placeholder`, `aria-label`).
- Les sélecteurs de genre et de tri sont des `<select>` associés à un libellé accessible.
- Le champ de priorité de la page Ma liste est un `<input type="number">` borné par `min` et `max`.
- Tout champ est associé à un libellé, visible ou accessible.

Le head de chaque page

- **charset**, **viewport**, **title** et **meta description** sont renseignés sur les trois pages.
- Les balises Open Graph de la page catalogue sont renseignées : titre, description et image de partage.

Partie 2 : Mise en forme, responsive et animations (CSS3)

Système de design

- Un bloc `:root` centralise la palette (primaire, secondaire, fond, texte, erreur), une échelle d'espacement, les rayons de bordure et la typographie. Aucune de ces valeurs n'est répétée en dur ailleurs dans les feuilles de style.
- La grille du catalogue est construite en CSS Grid ou en Flexbox et se réorganise sans casse lorsque le nombre de séries affichées varie.
- Les affiches ont un format constant : une image manquante ou de ratio différent ne doit pas déformer la carte.

Paliers responsive

Palier	Largeur	Grille du catalogue
Mobile	moins de 768 px	1 colonne
Tablette	768 à 1024 px	2 colonnes
Desktop	plus de 1024 px	3 colonnes ou plus

L'approche attendue est mobile first : la feuille de style décrit d'abord la version étroite, puis enrichit avec des media queries en `min-width`.

Animations et retours visuels

- Un indicateur de chargement est visible pendant chaque requête réseau et disparaît à la réception des données.
- Au moins deux transitions au survol sont mises en place, par exemple sur la carte série et sur le bouton d'ajout à la liste.
- La préférence système `prefers-reduced-motion` est respectée : un bloc `@media` neutralise les animations lorsqu'elle est active.

Les quatre états de l'interface

Chaque zone alimentée par le réseau doit être dessinée dans ses quatre états :

- **Chargement** : un indicateur visible, jamais un écran figé.
- **Succès** : les données affichées.
- **Vide** : aucun résultat, avec une indication de ce que l'utilisateur peut faire ensuite.
- **Erreur** : un message clair et un moyen de réessayer.

Partie 3 : Dynamisation et consommation de l'API (JavaScript)

Couche d'accès aux données

- Un module `js/api.js` regroupe tous les appels réseau. Il expose `getShows()`, `getShowById(id)`, `searchShows(texte)` et `sendSelection(payload)`, écrites en `async/await` et reposant sur `fetch`. Une constante unique `y` porte l'URL de base de l'API.
- Le code distingue trois situations : erreur réseau, réponse d'erreur HTTP (4xx / 5xx) et succès.
- Le code est réparti entre `api.js`, `utils.js` et `main.js`, chargés en `<script type="module">`, sans logique dupliquée d'une page à l'autre.

Rappel important

Une réponse 404 ou 500 ne déclenche pas le bloc `catch` : la promesse est tenue, le serveur a répondu. C'est à vous de tester `response.ok` et de lever l'erreur vous-même.

Page catalogue

- Au chargement, la liste des séries est récupérée puis les cartes sont construites dans le DOM.
- La liste des genres proposée dans le filtre est déduite dynamiquement des données reçues : aucun genre n'est écrit en dur dans le HTML.
- La recherche filtre les séries sur leur titre en temps réel, sans distinction de casse, et s'appuie sur un *debounce*.
- Le tri est réalisé côté client sur au moins deux critères : la note et l'année de première diffusion, chacun dans les deux sens.
- Le nombre de résultats affichés est annoncé dans une région `aria-live`.
- Les cas limites sont traités : aucune série ne correspond à la recherche, l'API est indisponible, l'affiche d'une série ne se charge pas, la note est absente.

Fiche détaillée

- L'identifiant est lu dans l'URL via `URLSearchParams`, puis la fiche est chargée.
- Trois cas fautifs sont gérés : identifiant absent de l'URL, non numérique, ou inexistant dans le catalogue.

- La note `rating.average` est rendue sous forme d'étoiles, accompagnée d'un équivalent textuel accessible.
- Le résumé, les genres, le statut, la chaîne de diffusion et l'année sont affichés. Une valeur manquante affiche une mention explicite, jamais la chaîne « null » ni un espace vide.

Ma liste

- Le bouton « Ajouter à ma liste » alimente une liste conservée côté client et survivant au changement de page ; le compteur du menu reflète son contenu sur les trois pages.
- La page Ma liste permet de modifier la priorité d'une série et de supprimer une ligne, et affiche le nombre total de séries retenues.
- La priorité est validée avant l'envoi : nombre entier, compris entre 1 et 10.
- L'envoi de la sélection effectue un `POST` avec la charge utile attendue, puis affiche un message de confirmation ou d'erreur.

Partie 4 : Accessibilité, qualité et livrables

Accessibilité et compatibilité

- Au moins cinq mesures d'accessibilité sont mises en oeuvre parmi : alternatives textuelles des affiches, annonce du nombre de résultats via une région `aria-live`, visibilité du focus clavier, contrastes, navigation clavier complète du menu et des filtres, libellés de formulaire correctement associés, respect de `prefers-reduced-motion`.
- Le rendu est vérifié sur au moins trois navigateurs parmi Chrome, Firefox, Edge et Safari ; les écarts constatés et leur traitement sont notés dans le README.
- Un audit Lighthouse (ou équivalent) est réalisé sur la page catalogue ; le score d'accessibilité obtenu et deux pistes d'amélioration sont reportés dans le README.

Organisation du code

L'arborescence livrée sépare le balisage, les feuilles de style, les scripts et les ressources statiques :

```
streamflix/  
├─ index.html  
├─ serie.html  
├─ liste.html  
├─ css/  
│   └─ style.css  
│   └─ responsive.css  
├─ js/  
│   └─ api.js  
│   └─ main.js  
│   └─ utils.js  
└─ assets/
```

| └─ images/

└─ README.md

- Les conventions de nommage sont respectées : camelCase en JavaScript, kebab-case en CSS.
- Le code est indenté, commenté aux endroits non évidents, et exempt de duplication manifeste.
- La console du navigateur ne contient ni erreur ni avertissement provenant de votre code.

Livrables

- **Lien GIT vers votre projet**
- Un fichier **README.md** comprenant : présentation du projet, prérequis, procédure de lancement, arborescence, endpoints consommés, limites connues, relevé des tests navigateurs et résultat de l'audit d'accessibilité.

Utilisation de l'IA générative

L'utilisation d'outils d'IA générative (ChatGPT, Claude, Copilot, Gemini, etc.) est autorisée pendant cette épreuve. Cependant, elle doit être systématiquement transparente et justifiée.

La transparence sur l'usage de l'IA fait partie intégrante de la notation.

Règles à respecter impérativement

- Si vous utilisez l'IA pour vous aider (débuguer, générer du code, rappeler un concept, rédiger un paragraphe, etc.), vous devez le signaler explicitement dans votre code à l'endroit concerné.
- Pour chaque utilisation, précisez : le fournisseur et le modèle utilisés (ex. : ChatGPT GPT-4o, Claude Sonnet, GitHub Copilot...), le prompt exact que vous avez soumis, et la réponse obtenue (ou un résumé si elle est longue).
- Expliquez pourquoi vous avez choisi d'utiliser l'IA plutôt qu'une recherche classique (documentation, Stack Overflow, cours). Exemples de justifications acceptées : gain de temps sur un problème bloquant, reformulation d'un concept mal compris, génération d'un squelette de code à adapter.
- Indiquez si vous avez modifié ou adapté la réponse de l'IA, et si oui, en quoi et pourquoi.

Ce qui sera valorisé

- Une utilisation pertinente et ciblée de l'IA (pas du copier-coller intégral sans recul)
- La transparence totale : mieux vaut déclarer une utilisation que de la masquer
- La capacité à expliquer et critiquer la réponse de l'IA (montrer que vous comprenez)

Ce qui sera pénalisé

- L'utilisation non déclarée d'IA détectée lors de la correction
- Le copier-coller brut d'une réponse IA sans adaptation, sans explication et sans recul critique
- L'incapacité à expliquer ou défendre un élément de votre rendu produit par IA
- Du code livré contenant des appels à des bibliothèques que le sujet interdit, symptôme

d'une génération non relue

Conseil : vous pouvez dédier une sous-section « Utilisation de l'IA » dans le readme, ou annoter directement chaque passage concerné ou même de produire un document à part. L'important est que le correcteur puisse identifier clairement ce qui vient de vous et ce qui vient d'un outil.

Pour aller plus loin

Ces extensions ne sont pas demandées. Abordez-les seulement une fois les quatre parties terminées et vérifiées.

- Apparition progressive des cartes au défilement, à l'aide de l'API `IntersectionObserver`.
- Recherche côté serveur via `GET /search/shows?q=`, avec comparaison du ressenti par rapport au filtrage côté client.
- Chargement parallèle de la série et de ses épisodes avec `Promise.all` sur la fiche détaillée.
- Thème clair et thème sombre, basculés par un bouton et mémorisés d'une visite à l'autre.
- Pagination ou défilement infini sur le catalogue.
- Mise en cache des réponses de l'API pour éviter de recharger les mêmes données à chaque navigation.

Checklist de relecture

Réservez les dix dernières minutes à cette relecture. Ne codez plus, vérifiez.

- ☐ Le site se lance depuis un dossier propre, sans installation ni compilation.
- ☐ Aucune donnée de série n'est écrite en dur dans le HTML.
- ☐ L'URL de base de l'API n'apparaît qu'à un seul endroit du code.
- ☐ Chaque zone alimentée par le réseau possède ses quatre états : chargement, succès, vide, erreur.
- ☐ L'API coupée affiche un message clair, pas une page vide.
- ☐ Une recherche sans résultat affiche un état vide explicite.
- ☐ Le site tient à 360 px de large sans défilement horizontal.
- ☐ La navigation complète fonctionne au clavier et le focus reste visible.
- ☐ Aucune couleur ni espacement n'est répété en dur dans le CSS.
- ☐ Un identifiant absurde dans l'URL de la fiche affiche une erreur propre.
- ☐ La console du navigateur est vide d'erreurs.
- ☐ Le README explique comment lancer le site et ce qui reste incomplet.

Annexe : ressources

- **MDN Web Docs** : <https://developer.mozilla.org>
- **TVMaze API** : <https://www.tvmaze.com/api>
- **Can I use, compatibilité navigateurs** : <https://caniuse.com>
- **Outils conseillés** : VS Code (Prettier, ESLint, Live Server), les DevTools du navigateur, Lighthouse.